

Learning Programming through Data Visualization: Developing Inclusive Materials for Genders and Fields, and Practicing in an Art and Design Course

Takehisa Mashimo
Seian University of Art and Design
Otsu, Shiga, Japan
mashimo@seian.jp

Taeko Ariga
Doshisha Women's College of Liberal
Arts
Kyoto, Kyoto, Japan
tariga@dwc.doshisha.ac.jp

Tomoko Yoshida
Kyoto Notre Dame University
Kyoto, Kyoto, Japan
tyoshida@notredame.ac.jp

Abstract

This study developed learning materials for introductory computer programming from the viewpoint of gender and field inclusion. Subjects often used as learning materials for programming, like robotics and electronics, do not engage female students and students studying the humanities. We have considered the strategy of gender-inclusive and field-inclusive programming education, with a focus on data visualization. Finding interesting data and expressing it in an informative way using programming would enhance learners' motivation regardless of their major. We have developed an original library (Datamate.js) for data visualization, web content for classroom instruction and self-study, and a layout tool for publishing data visualization results on a web page. In this paper, we report on the library and the tool we developed, as well as our classroom practices at an art university.

ACM Reference Format:

Takehisa Mashimo, Taeko Ariga, and Tomoko Yoshida. 2025. Learning Programming through Data Visualization: Developing Inclusive Materials for Genders and Fields, and Practicing in an Art and Design Course. In *SIGGRAPH Asia 2025 Educator's Forum (SA Educator's Forum '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3757372.3771855>

1 Introduction

Statistics show that men still comprise the majority of individuals studying and working in the IT field in Japan and worldwide, although opportunities to learn IT skills are equally available regardless of gender. This situation persists despite the absence of evidence that gender differences in information/mathematical competence exist [Lau and Yuen, 2009; McBroom et al., 2020; McAdams, 2018]. The reason is that some unconscious biases hinder women from studying computer science (CS) or joining the IT industry. Corneliussen and Tveranger pointed out that a gender-blind strategy alone, without an inclusive approach in schools, has reinforced the social gender imbalance in the IT field [Corneliussen and Tveranger, 2018].

Further, programming education has often focused on subjects related to electronics, robotics, and mathematics, resulting in a substantial accumulation of learning materials in that domain. Relying solely on these materials may diminish female students' interest in learning programming and exacerbate the gender imbalance.

This is not just a gender perspective issue. Since students studying CS are predominantly male and students in art and design are predominantly female, enhancing gender inclusion is directly connected to strengthening the inclusion of the humanities field. Graßl et al. researched programming education using a diversity-enhancing approach for various students with different interest levels in CS and diverse social backgrounds [Graßl et al., 2022]. They reported that avoiding non-CS tasks and mathematics had a positive effect on both female and non-CS students. Gender-inclusive materials in programming education contribute directly to field inclusion.

Therefore, it is essential to employ a specific approach that engages all students, regardless of gender, who are not currently interested in CS to recognize the value of learning CS for their field of study. We have been researching programming education, using data visualization as a subject matter to create gender-inclusive materials. This purpose is not simply to increase the number of "science" girls; it also leads to field inclusion. Students can utilize programming in areas of personal interest to broaden the scope of their study and research, whether they study the sciences or the humanities. This paper reports on the practice of media content production classes that use our learning materials at an art university.

2 Design of the Learning Materials

2.1 Data visualization as subject matter

The subject matter and content addressed in programming education significantly affect motivation and familiarity. Several studies have reported elements of learning materials that encourage positive attitudes, particularly among female learners. McAdams found that girls ages 8 and 9 enjoyed creating programs that incorporate visual design, narrative, and problem-solving elements [McAdams, 2018]. Aivaloglou and Hermans identified different preferences for project types based on gender in after-school code clubs. Girls often favored storytelling and creative and visual exercises, whereas boys tended to prefer implementing games [Aivaloglou and Hermans, 2019]. McLean and Harlow examined the effect of integrating aspects of social relevance, storytelling, and design tasks into activities in a study on STEM activities. They reported that incorporating

elements of social relevance and design tasks into STEM play can help engage more girls [McLean and Harlow, 2017]. In a survey of college and university students, Doubé and Lang reported that female students could be motivated to learn technical concepts by association with domains considered creative, fashionable, and sociable, as well as those not perceived as dominated by masculine stereotypes [Doubé and Lang, 2012]. Poster, Hislop, and Ellis concluded that examples and assignments with a social impact in early computing courses were often more interesting and motivating, especially for female students, compared to those that focus on business [Postner et al., 2023].

In other words, these studies indicate that the following factors are essential when choosing a subject:

1. Students can incorporate social issues.
2. The result of programming is useful for students and society.
3. Students can demonstrate their creativity.

We have developed learning materials, specifically programs for interactive data visualization. Students can find and handle data linked to realistic themes that interest them, regardless of gender biases and in any field of study. The subject of visualizing quantitative information is also used to teach mathematical concepts to students with diverse interests [Friend et al., 2023]. Visualizing data as subject matter can be considered gender-neutral and field-neutral for all learners. Infographics created through coding can offer an intuitive and engaging experience that stimulates the viewer's curiosity. In this sense, they become "useful" for delivering information in an impactful way. Consequently, creating such infographics has the potential to enhance motivation among students from various academic backgrounds who are learning programming.

2.2 Programming environment and outline of learning materials

We chose JavaScript as the programming environment because it is a leading programming language for creating web content and easily facilitates publishing programming results on a web browser. A variety of useful libraries for JavaScript have been developed. We use the p5.js library to teach fundamental programming concepts through drawing graphical primitives. The p5.js library is well known as based on Processing [Reas et al., 2007], which has numerous convenient functions for drawing and animating graphical primitives.

Several development editors are available for programming with JavaScript and p5.js, including the p5.js web editor, OpenProcessing, Processing standalone, and general-purpose text editors. In our research, we have used either Processing Standalone or the p5.js web editor in classes. A local web server is needed to display the results of the data visualization program on a web page. Processing standalone provides both an editing function and an internal local server to preview the program's results. This helps classes avoid constructing a complicated server environment.

Our teaching materials consist of the following three components:

1. The original library (Datamate.js) to create data visualization easily.
2. Web content for both classroom instruction and self-study.

3. The layout tool for creating Web pages to publish data visualization results.

While the p5.js library facilitates drawing graphics, data visualization requires many programming processes beyond the functions it provides. These processes include reading external files, parsing and normalizing data, and linking motion graphics to data, which can be difficult for beginners to understand. To address this issue, we have developed an original library called Datamate.js.

We have also developed web-based learning materials that include basic programming examples and exercises focused on drawing primitive graphics with data using Datamate.js. Additionally, these web-based materials deliver advanced sample programs demonstrating how to visually communicate data insights through motion and interactive graphics with our library.

Our programming education aims not only to complete a program in the classroom but also to publish the resulting data visualization on a web page. This is why we selected JavaScript as our programming language. However, students who have not yet learned HTML or CSS encounter another challenge in formatting and embedding program outputs into an appropriate web layout. To solve this hurdle, we developed a layout tool called MLUI (Multi Layout User Interface) that enables students to embed their program results into web pages without requiring any additional coding.

2.3 Original library - Datamate.js

The original library for data visualization provides the following major functions:

1. Specify data directly or import data from a CSV file, generating it in an internal table format.
2. Retrieve data sequentially from the internal table for visualization.
3. the display area and designate rows or columns in the internal table to be displayed in that area.
4. Maintain information on the position and size of the display area for drawing purposes.
5. basic rigid-body motion to link data in the internal table and animate.

Considering simplicity in implementation, we determined the signatures of the functions, which include names, arguments, and return values. Figure 1 shows an example of a student's work. The height of rice in a bowl represents the amount of rice consumed. This program first prepares data through the functions (1) described above and divides the display area into seven areas by the functions (3). Next, it draws a rice bowl and a heap of rice according to data obtained from the internal table through the functions (2) in each area. When the program draws them, it uses the size and position gained through the functions (4) without calculating them.

Animating graphic elements like circles or lines based on physical principles is a useful method for data visualization. However, writing a program to calculate their position and velocity is not easy for students who are not familiar with physics. Students can create programs that generate motion graphics in response to data using the functions (5).

There are JavaScript libraries for visualizing data, such as D3.js and plotly.js. However, their capabilities for data visualization are primarily limited to graphs and charts. Furthermore, these libraries

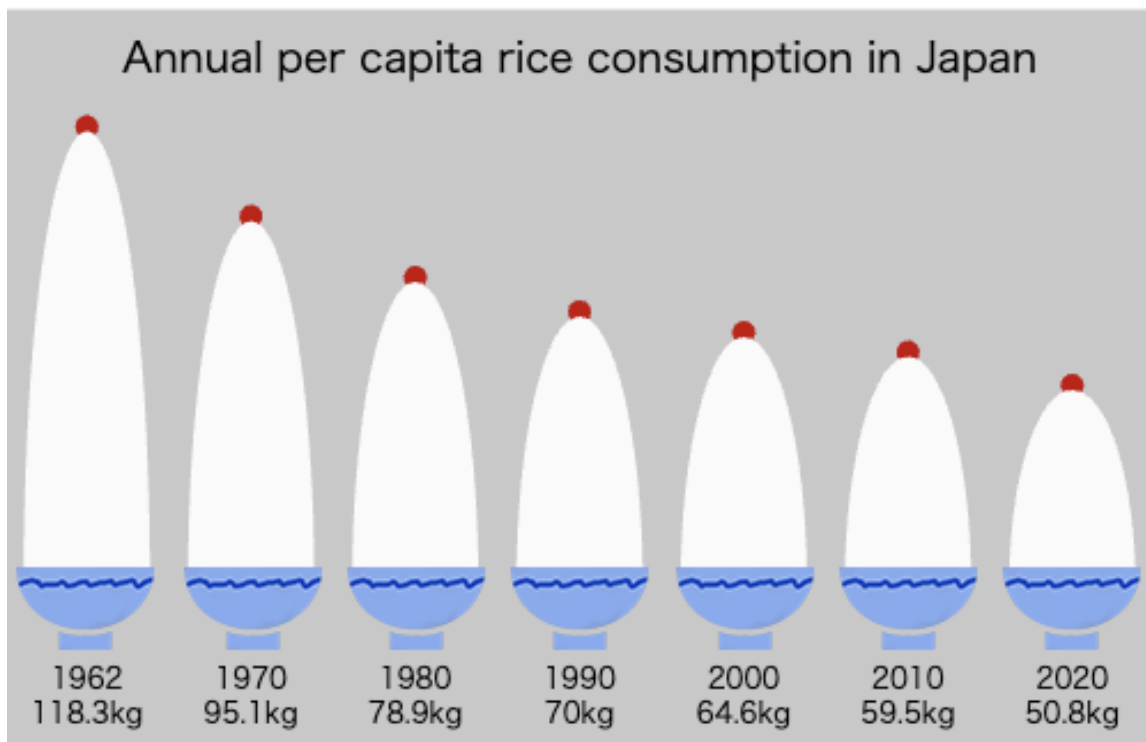


Figure 1: Example of a student's work (A heap of rice and a pickled plum show rice consumption)

require students to understand APIs, including the DOM (Document Object Model) and associative arrays. This often results in complex and lengthy code, which can hinder effective programming learning for beginner-level students. Our library aims to help novices understand the fundamentals of data visualization in the simplest way possible, thereby fostering their motivation in computer programming.

2.4 Multi-layout user interface - MLUI

MLUI (Multi-Layout User Interface) is a graphical user interface (GUI) tool that allows users to easily lay out text, images, and HTML canvas elements without writing any HTML or CSS code. MLUI has been developed using Electron, an open-source framework. Similar tools include Adobe XD and Figma; however, these are not designed to lay out canvas elements or work with p5.js and Datamete.js.

MLUI enables users to specify the size and position of text, image, and canvas elements through an intuitive panel interface. Users can easily integrate a canvas element with a JavaScript program by copying and pasting from an external editor, then quickly preview the program's results on a web page. Additionally, they can execute the web page in a separate window, which functions as a basic browser, to check the final appearance and display it in full-screen mode for digital signage. Furthermore, MLUI's build function exports HTML, CSS, and JavaScript files that can be uploaded to a web server and published online immediately.

Figure 2 shows the MLUI interface. Users select text, an image, or a canvas to add to a web page through the tool panel. These

elements can then be positioned on the layout panel by dragging and dropping. The node tree displays the parent-child relationship, and the order of elements placed on the layout panel, and users can interactively modify these elements with a mouse. The inspector panel allows users to view and specify the size, position, and other attributes of the elements. The bottom of the MLUI is Chrome's standard developer tool, where you can check for errors in the previewing canvas script.

3 practices in class for expression

Using the above learning materials, we conducted a 14-week course for second-year students in the Media Design Department at Seian University of Art and Design (eight female students and one male student in 2024). Classes are held once a week for 100 minutes. This course targets learning visual expression using computer programming. In class, students learn the basics of programming using the p5.js library during the first four weeks. In week five, they learn how to use MLUI, and in week six, they develop their own drawing application using MLUI as the midterm project.

Figure 3 presents an example developed by a student with MLUI. This application allows users to select basic shapes, such as a circle and a triangle, as well as colors by keyboard, and then draw with them as if using a paintbrush. After understanding how to create programs with p5.js, students use MLUI to lay out the canvas elements that execute their programs, format them, and complete their applications.

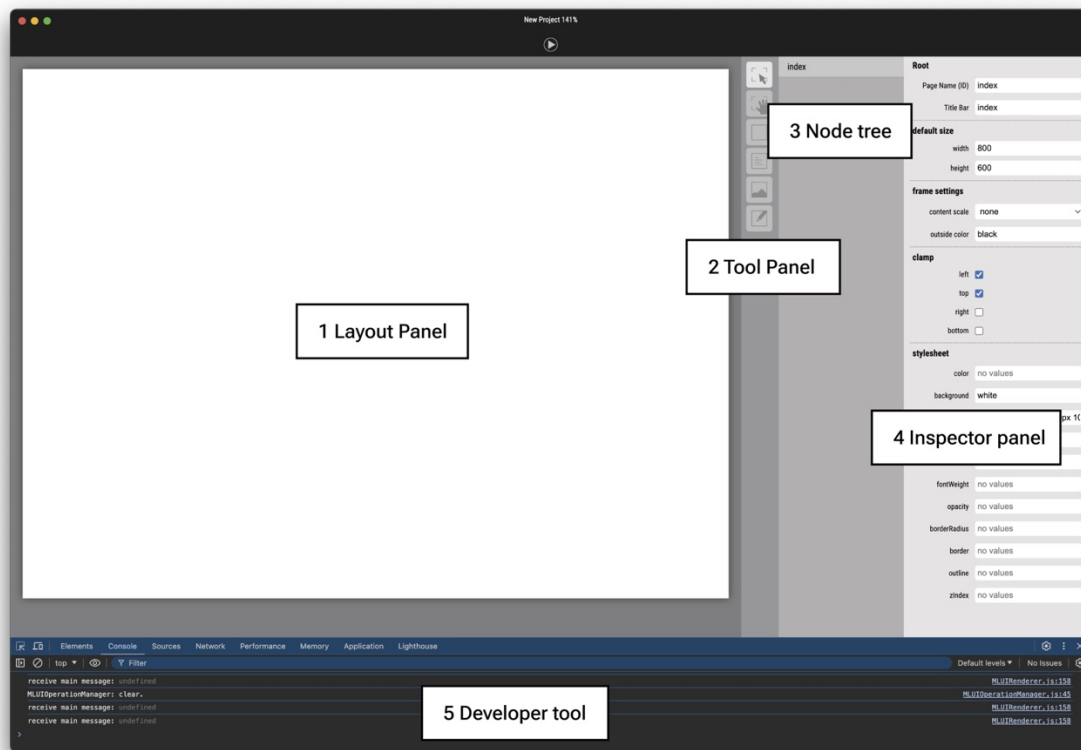


Figure 2: MLUI editor

In class, after placing text, images, and canvas elements using MLUI's graphical user interface, students connect their own programs to the canvas elements and then execute them. They check and adjust both the screen layout and the code's behavior. The left part of Figure 3 shows the title and description of the drawing application. Students can easily specify them using MLUI without needing to write HTML tags or CSS code. This feature helps students concentrate on their creativity when developing drawing programs.

After the midterm project, students learn data visualization by writing programs that represent data using the `datamete.js` library. For the final project, students spend the last five weeks developing a program to visualize data they find interesting and publish it as web content. Figure 4 displays a student's work of the final project that visualizes the monthly rainfall in Otsu City, Shiga Prefecture, for 2024 as the spread of falling rain.

MLUI can place multiple canvas elements on the screen and link each one to the same or different program code individually. It can also use a special parameter called `userdata` to assign values to the program codes connected to each canvas and reflect those values when the program is executed. In Figure 4, the same program for data visualization is used for 12 canvases, where the size of the spread of the dropping liquid image varies depending on the amount of precipitation. This MLUI feature simplifies a complex process involving multiple rendering steps while reflecting different

values with the same algorithm. As a result, students can focus solely on algorithms for visualizing numerical data. It ensures continuity in learning programming and enables the completion of an application.

Through this learning process, we aimed to give students an understanding of how programming can be applied to their further creations and to empower them to continue utilizing programs after completing the course.

4 DISCUSSION

Amiri emphasizes the importance of programming education in the art and design curriculum [Amiri, 2011]. Knoche and Patton also mentioned that creative coding can benefit studio art practice while making broader contributions to conceptions of computational thinking across disciplines [Knoche and Patton, 2015]. We have been practicing programming education in the field of media art production for over two decades. This study proposes an approach that differs from conventional programming education used in science and engineering and reports on its implementation.

Subjects commonly used as teaching materials for programming education, such as robotics and electronics, are not engaging for most students in the art and design fields, and female students. Moreover, the gender ratio of students in art and design departments in Japan is 70-80% female. Based on this background, we developed introductory programming learning materials focused

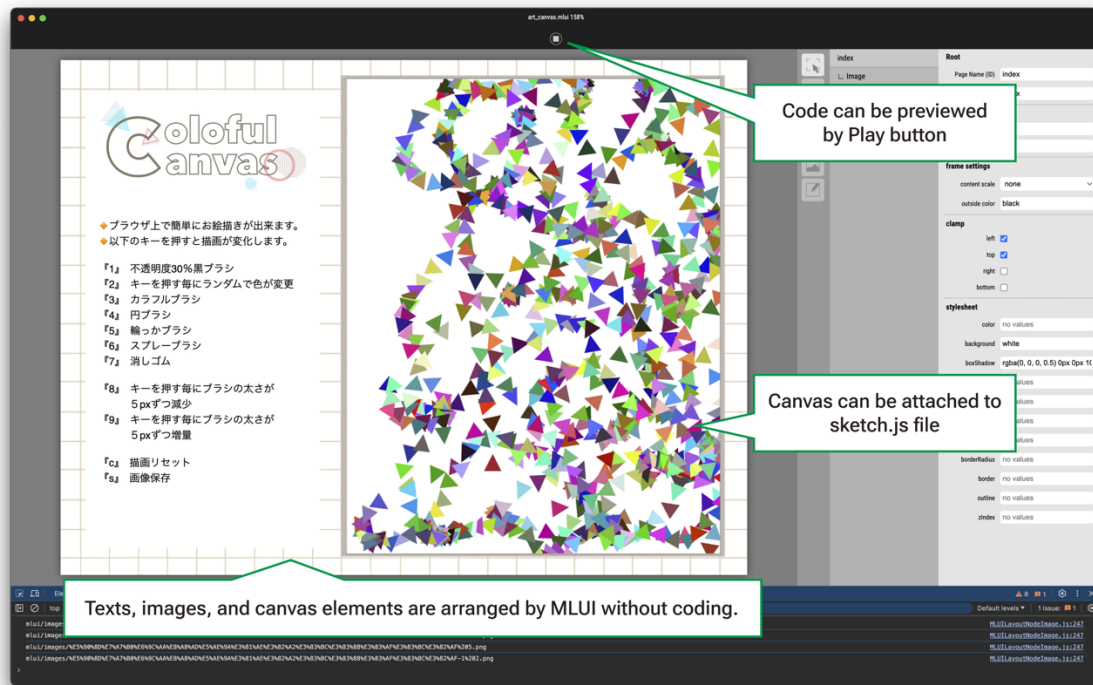


Figure 3: Preview of a student's drawing application on MLUI

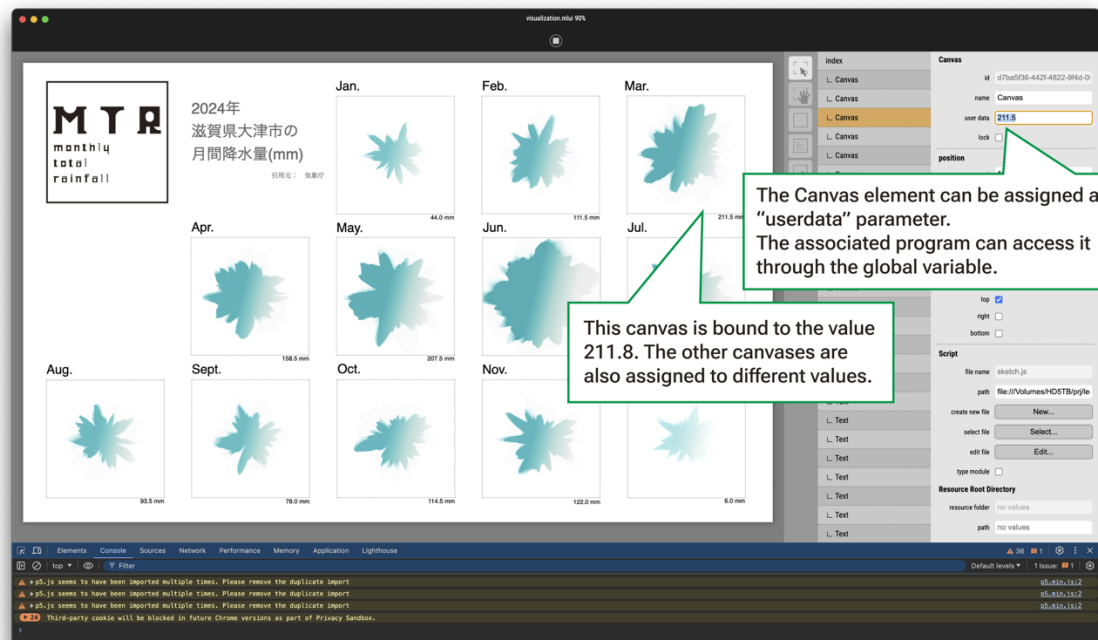


Figure 4: Preview of a student's data visualization on MLUI: the amount of precipitation

on data visualization, considering gender and field diversity. Data visualization as a programming learning topic can be an effective way to learn programming that is unbiased toward gender, field of study, or personal interests, thereby fostering creative engagement.

Through programming education using our learning material in regular classes at an art university, students not only acquired fundamental programming skills but also applied them to produce interactive media design works that integrated infographics and data visualization. This demonstrates that programming can go beyond being just a technical skill and become a vital tool for creative practice. We intend to continue researching and enhancing these learning materials in the future.

Acknowledgments

This work was supported by JSPS KAKENHI Grant Number JP21K02790.

References

- Wilfred W. F. Lau and Allan H. K. Yuen. 2009. Exploring the effects of gender and learning styles on computer programming performance: Implications for programming pedagogy. *British Journal of Educational Technology*, Vol.40, No.4, 696–712. <https://doi.org/10.1111/j.1467-8535.2008.00847.x>
- Jessica McBroom, Irena Koprinska, and Kalina Yacef. 2020. Understanding Gender Differences to Improve Equity in Computer Programming Education. *Proceedings of Twenty-Second Australasian Computing Education Conference*. 185–194. <https://doi.org/10.1145/3373165.3373186>
- Terence McAdams. 2018. Gender and computer programming: teaching and learning strategies designed to increase the engagement of girls. EdD Thesis, Institute of Education, University of Reading.
- Hilde G. Corneliussen and Fay Tveranger. 2018. Programming in secondary schools in Norway: a wasted opportunity for inclusion. *Proceedings of the 4th Conference on Gender & IT*, 175–182. <https://doi.org/10.1145/3196839.3196867>
- Iris GraBl, Michael Vierhauser, Barbara Sabitzer, Lisa Kuka, Alexander Hofer, and David Muster. 2022. Exploring diversity in introductory programming classes: an experience report. *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering Education and Training*, 102–112. <https://doi.org/10.1145/3510456.3514155>
- Efthimia Aivaloglou and Felienne Hermans. 2019. How is programming taught in code clubs? Exploring the experiences and gender perceptions of code club teachers. *Proceedings of the 19th Koli Calling International Conference on Computing Education Research*, 1–10. <https://doi.org/10.1145/3364510.3364514>
- Mandy McLean and Danielle Harlow. 2017. Designing Inclusive STEM Activities: A Comparison of Playful Interactive Experiences Across Gender. *Proceedings of the 2017 Conference on Interaction Design and Children*, 567–574. <https://doi.org/10.1145/3078072.3084326>
- Wendy Doubé and Catherine Lang. 2012. Gender and stereotypes in motivation to study computer programming for careers in multimedia. *Computer Science Education*, Vol. 22, No. 1, 63–78. <https://doi.org/10.1080/08993408.2012.666038>
- Lori Postner, Gregory W. Hislop, and Heidi J.C. Ellis. 2023. Humanitarian Applications Increase Interest and Motivation of Women in Computing. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V.1*, 416–422. <https://doi.org/10.1145/3545945.3569832>
- Michelle Friend, Andrew W. Swift, Betty Love, and Victor Winter. 2023. A Wolf in Lamb's Clothing: Computer Science in a Mathematics Course. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V.1*, 256–262. <https://doi.org/10.1145/3545945.3569736>
- Casey Reas, Ben Fry, and John Maeda. 2007. *Processing: A Programming Handbook for Visual Designers and Artists*. The MIT Press.
- Faramarz Amiri. 2011. Programming as Design: The Role of Programming in Interactive Media Curriculum in Art and Design. *International Journal of Art & Design Education*, Vol. 30, Issue 2, 200–210. <https://doi.org/10.1111/j.1476-8070.2011.01680.x>
- Aaron D. Knochel, and Ryan M. Patton. 2015. If Art Education Then Critical Digital Making: Computational Thinking and Creative Code. *Studies in Art Education*, Vol. 57, No. 1, 21–38. <https://www.tandfonline.com/doi/abs/10.1080/00393541.2015.11666280>